

Document on observability

Observability Metrics:

Observability metrics are numerical data that help monitor the performance and health of applications, systems, and infrastructure. These metrics provide insights into how a system behaves in real time and help engineers detect problems, troubleshoot issues, and improve system reliability. Observability mainly uses three types of telemetry data: **metrics, logs, and traces**, where metrics represent measurable values such as CPU usage, request count, response time, and error rates.

- **Metrics** are numerical values that show system performance, such as CPU usage, memory utilization, request count, and response time.
- **Logs** are detailed records of events that occur within a system, helping identify errors, warnings, or system activities.
- **Traces** track the complete path of a request as it moves through different services in a distributed system, helping identify performance bottlenecks.

Observability metrics are commonly categorized into three types:

- **Application metrics** – measure application performance such as request latency, response time, and error rate.
- **Infrastructure metrics** – monitor system resources like CPU usage, memory consumption, disk I/O, and network traffic.
- **Business metrics** – track business-related activities such as number of users, transactions, or revenue generated.

What is observability?

The ability to understand what is happening inside a system by looking at its outputs.

Why observability?

It helps to find the root cause fast and improves reliability.

Golden Signals:

In Site Reliability Engineering (SRE), thousands of metrics can exist, but focusing on the most important ones helps simplify monitoring. The **Golden Signals** are four key metrics used to measure system reliability and performance.

1. Latency

Latency measures the **time taken for a system to respond to a request**. High latency indicates slow system performance and poor user experience.

2. Traffic

Traffic represents the **amount of demand on a system**, usually measured as requests per second or number of transactions.

3. Errors

Errors measure the **rate of failed requests or system failures**, such as HTTP errors or application exceptions.

4. Saturation

Saturation indicates **how close a system is to reaching its resource limits**, such as high CPU usage, memory consumption, or queue length.